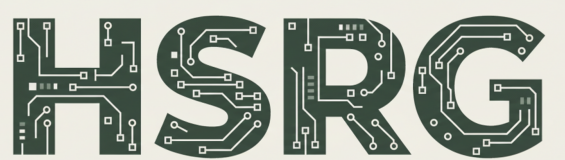


GPU Kernel Programming

Harvard Systems Reading Group (HSRG)

Emmanuel Rassou . 02/07/26



Isn't the hardware fast enough already?!

Motivation



Erik Bernhardsson  

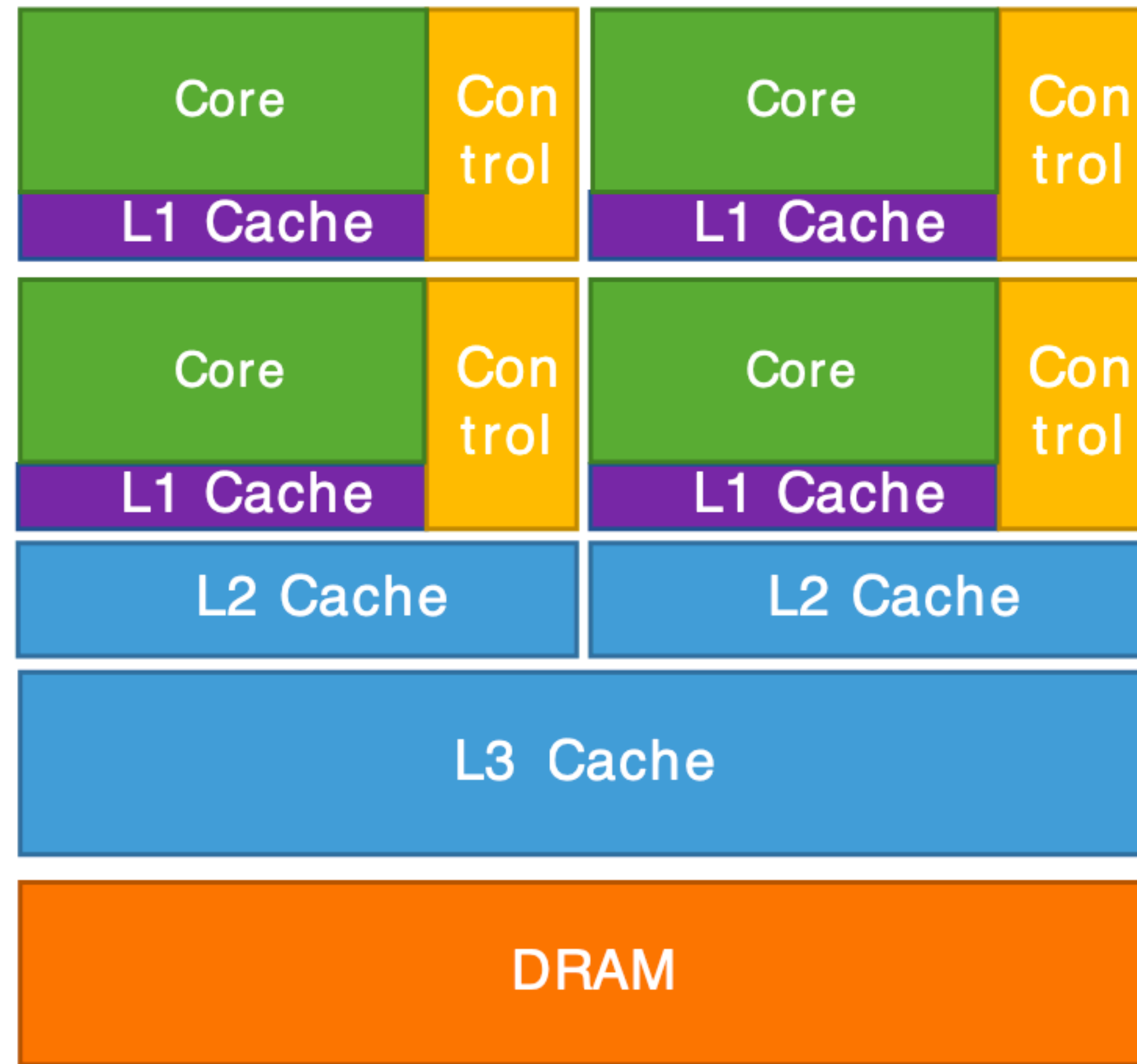
@bernhardsson



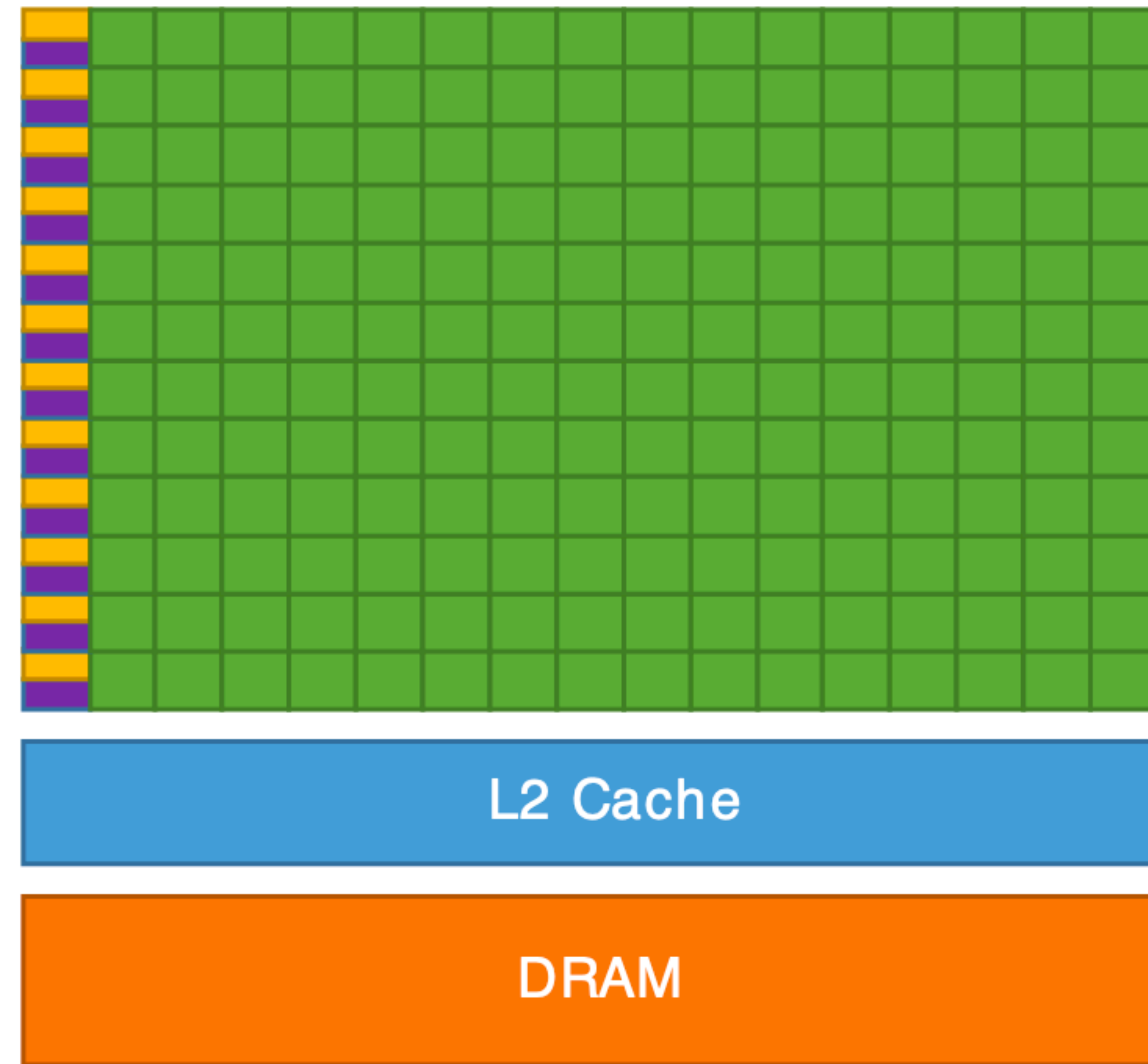
It's ironic that Blackwells have been out since 2024 but people still prefer Hoppers because the kernels aren't Blackwell-optimized yet, and now the Hopper prices are going up.

7:19 PM · Jan 23, 2026 · **116.7K** Views

CPU vs GPU



CPU

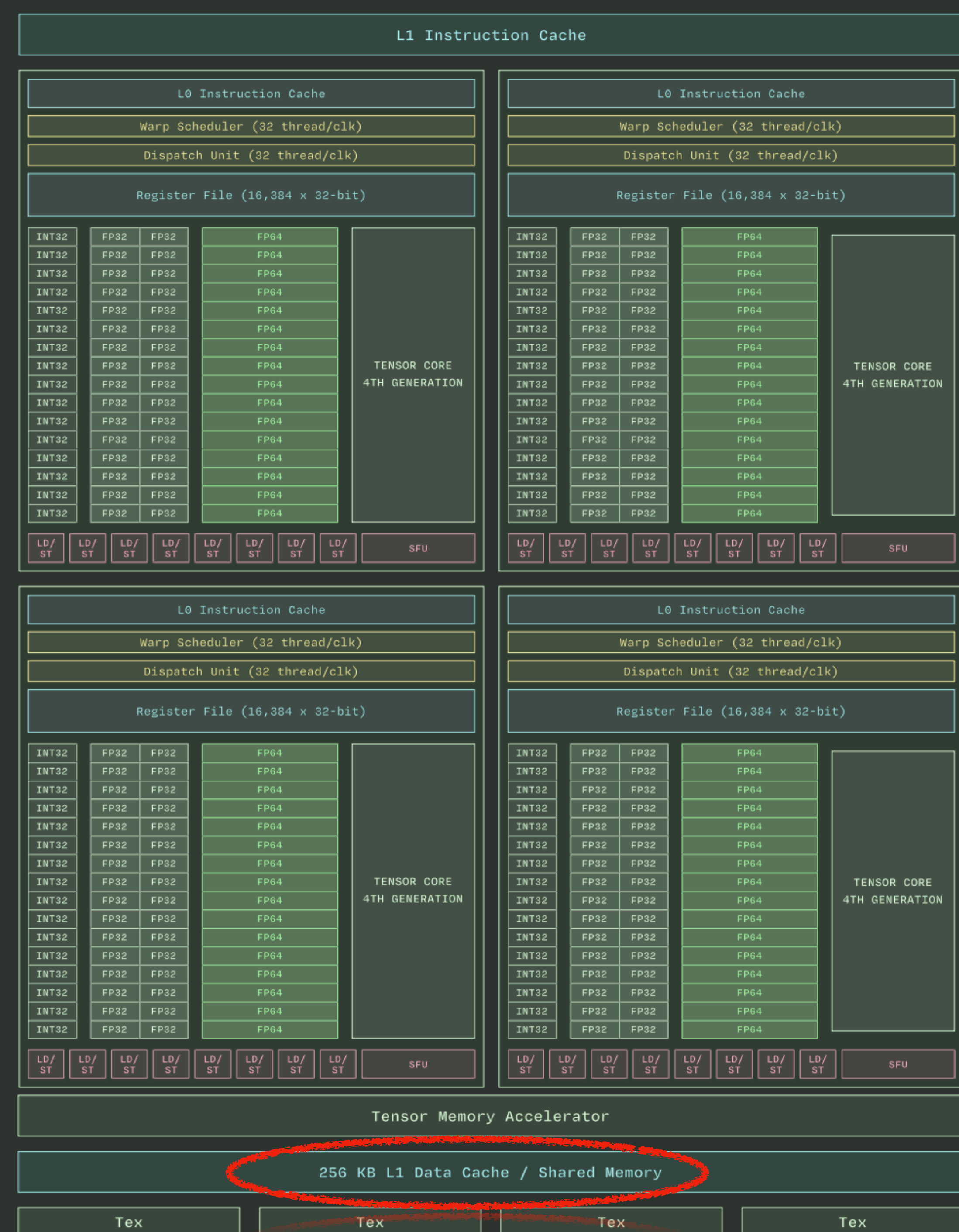


GPU

What is an SM?

Streaming Multiprocessor (on H100)

- GPU SM $\approx$ 1 CPU core
 - But no speculative execution
- Standard version of H100 has 132 SMs
- GPU Core $\approx$ CPU pipe
- 128 Cuda cores $\Rightarrow$ scalar/vector math
- 4 Tensor cores $\Rightarrow$ matrix math



Shared Memory

Aka L1 data cache

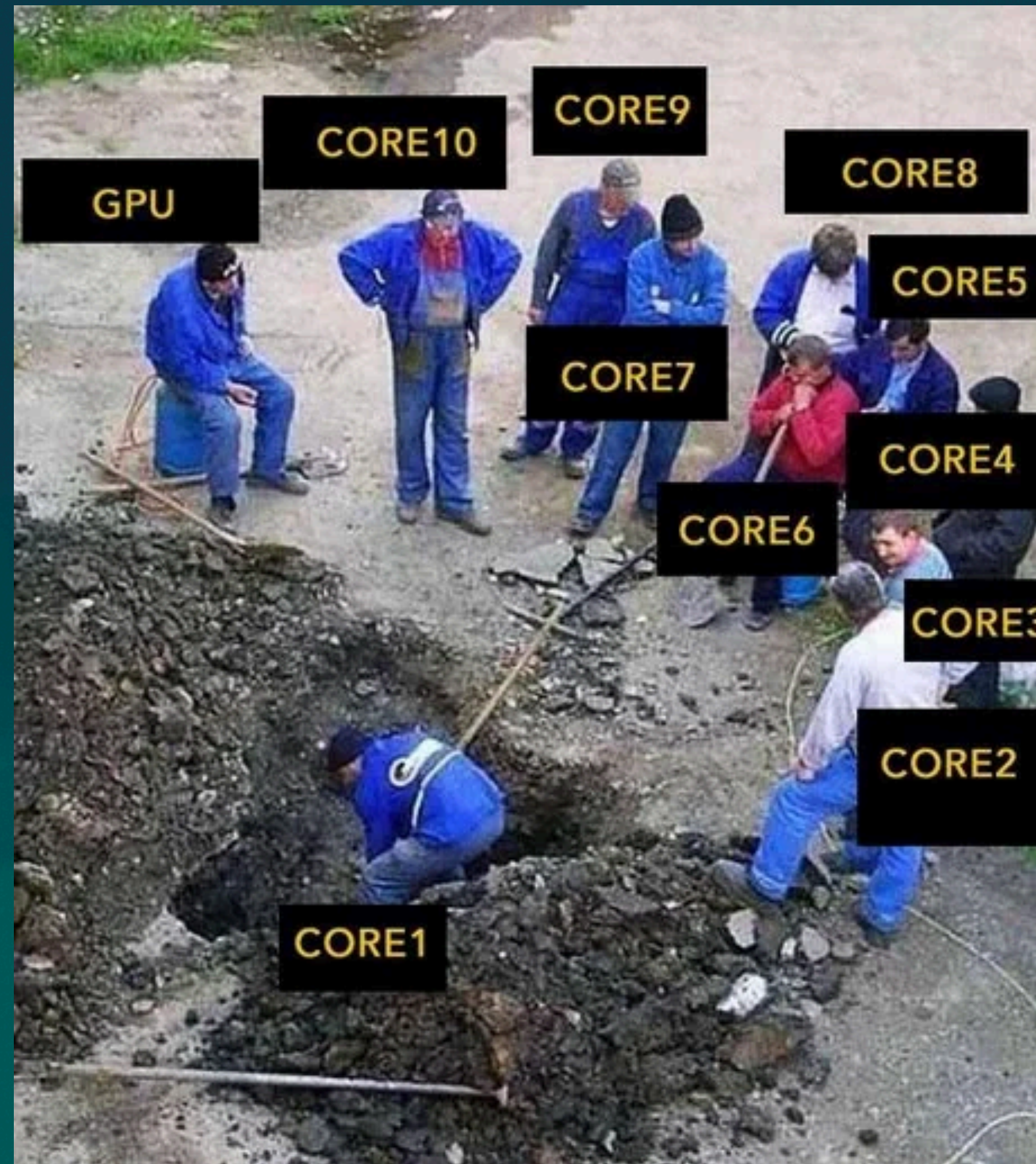
- Smaller but faster than global memory (across SMs)
- Corresponds to a **thread block**
 - 1 thread block = blockDim no. of threads
 - `__syncthreads()` only syncs within the block



CUDA:
`threadIdx.x`

`blockIdx.x`

Time to be a systems programmer



Introducing KernelBench Problems

KernelBench: Can LLMs Write Efficient GPU Kernels?

Anne Ouyang^{1,*}, Simon Guo^{1,*}, Simran Arora¹, Alex L. Zhang², William Hu¹, Christopher Ré¹, and Azalia Mirhoseini¹

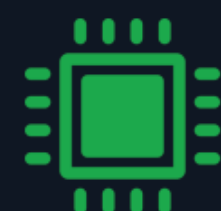
¹Stanford University

²Princeton University

February 18, 2025

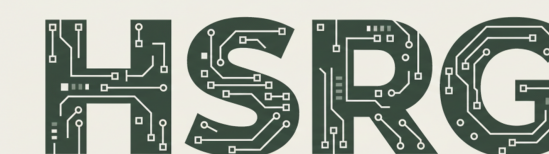
Abstract

Efficient GPU kernels are crucial for building performant machine learning architectures, but writing them is a time-consuming challenge that requires significant expertise; therefore, we explore using language models (LMs) to automate kernel generation. We introduce **KernelBench**, an open-source framework for evaluating LMs' ability to write fast and correct kernels on a suite of 250 carefully selected PyTorch ML workloads. KernelBench represents a real-world engineering environment and making progress on the introduced benchmark directly translates to faster practical kernels. We introduce a new evaluation metric fast_p , which measures the percentage of generated kernels that are functionally correct and offer a speedup greater than an adjustable threshold p over baseline. Our experiments across various state-of-the-art models and test-time methods show that frontier reasoning models perform the best out of the box but still fall short overall, matching the PyTorch baseline in less than 20% of the cases. While we show that results can improve by leveraging execution and profiling feedback during iterative refinement, KernelBench remains a challenging benchmark, with its difficulty increasing as we raise speedup threshold p .



tensara

LeetGPU



Cuda Kernel Template

Host fn:

```
const int BLOCKDIM = "...";

void solution(...) {

    dim3 blockDim = ...

    dim3 gridDim = ...

    kernel<<gridDim,blockDim>>(args);

    cudaDeviceSynchronize();
```

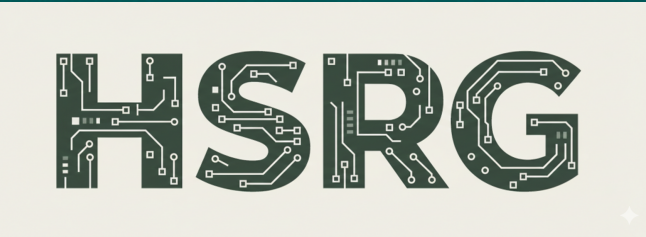
Kernel fn:

```
__global__ void kernel(args) {

    int i = blockIdx * blockDim + threadIdx;

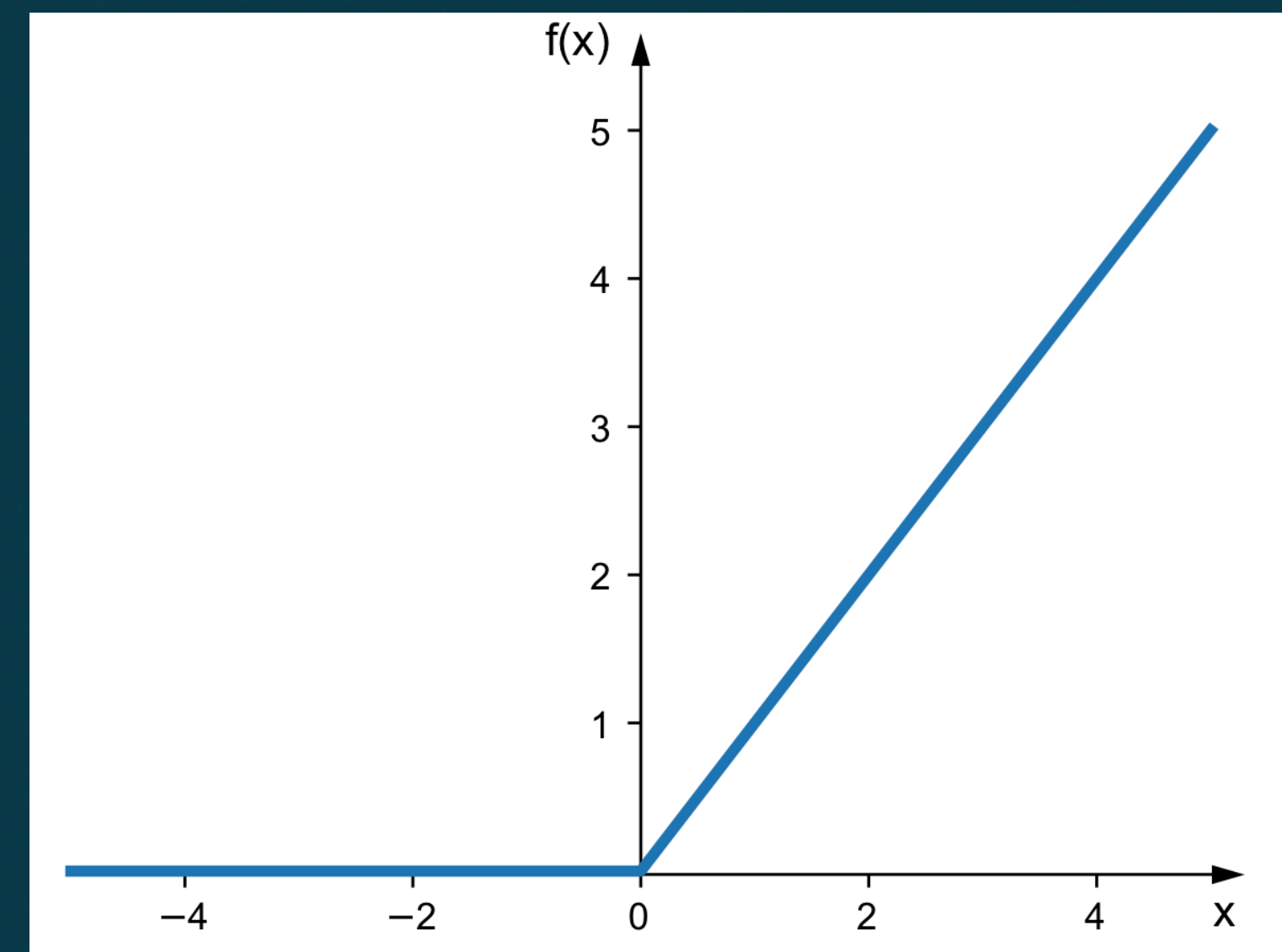
    ... }
```

Fn qualifier	Called from	Runs on
__host__	CPU	CPU
__global__	CPU	GPU
__device__	GPU	GPU



Problem #1: ReLU

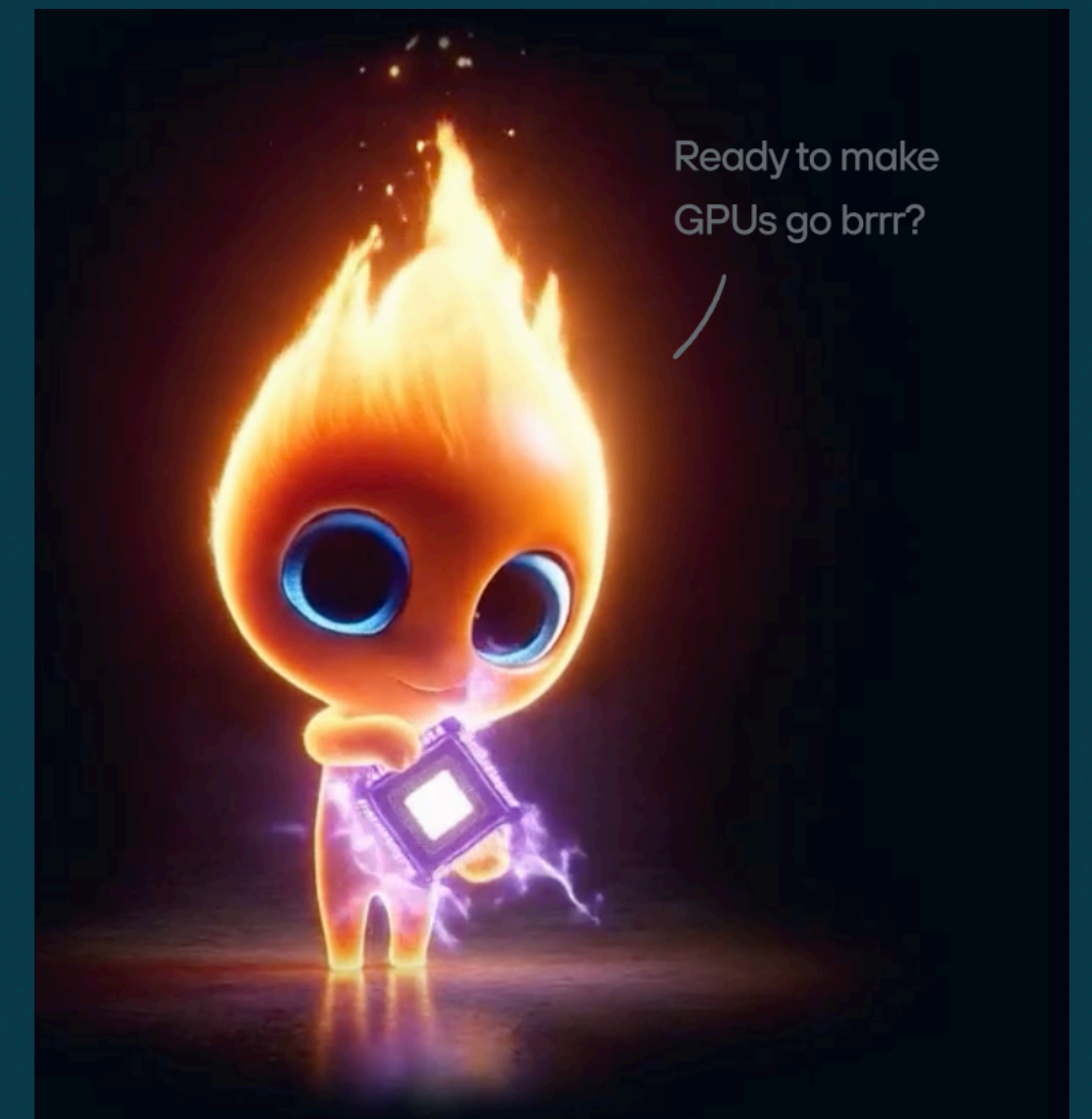
<https://tensara.org/problems/relu>



- Tune blockDim - does the optimal value depend on the GPU generation?
- Try use a max function

Aside: Mojo

GPU Programming but Pythonic



Source: <https://www.modular.com/mojo>

```
1  fn add(out: &mut LayoutTensor, a: &LayoutTensor, b: &LayoutTensor):  
2      i = global_idx.x  
3      if i < size:  
4          out[i] = a[i] + b[i]
```

Source: <https://www.modular.com/mojo>

Problem #2: GEMM

<https://tensara.org/problems/matrix-multiplication>

```
output[i][j] += input[i][k] * input[k][j] for k = 0, 1, ...
```

$$\begin{bmatrix} a_1 & a_2 & a_3 \\ a_4 & a_5 & a_6 \\ a_7 & a_8 & a_9 \end{bmatrix} \begin{bmatrix} b_1 & b_2 & b_3 \\ b_4 & b_5 & b_6 \\ b_7 & b_8 & b_9 \end{bmatrix} = \begin{bmatrix} c_1 & c_2 & c_3 \\ c_4 & c_5 & c_6 \\ c_7 & c_8 & c_9 \end{bmatrix}$$

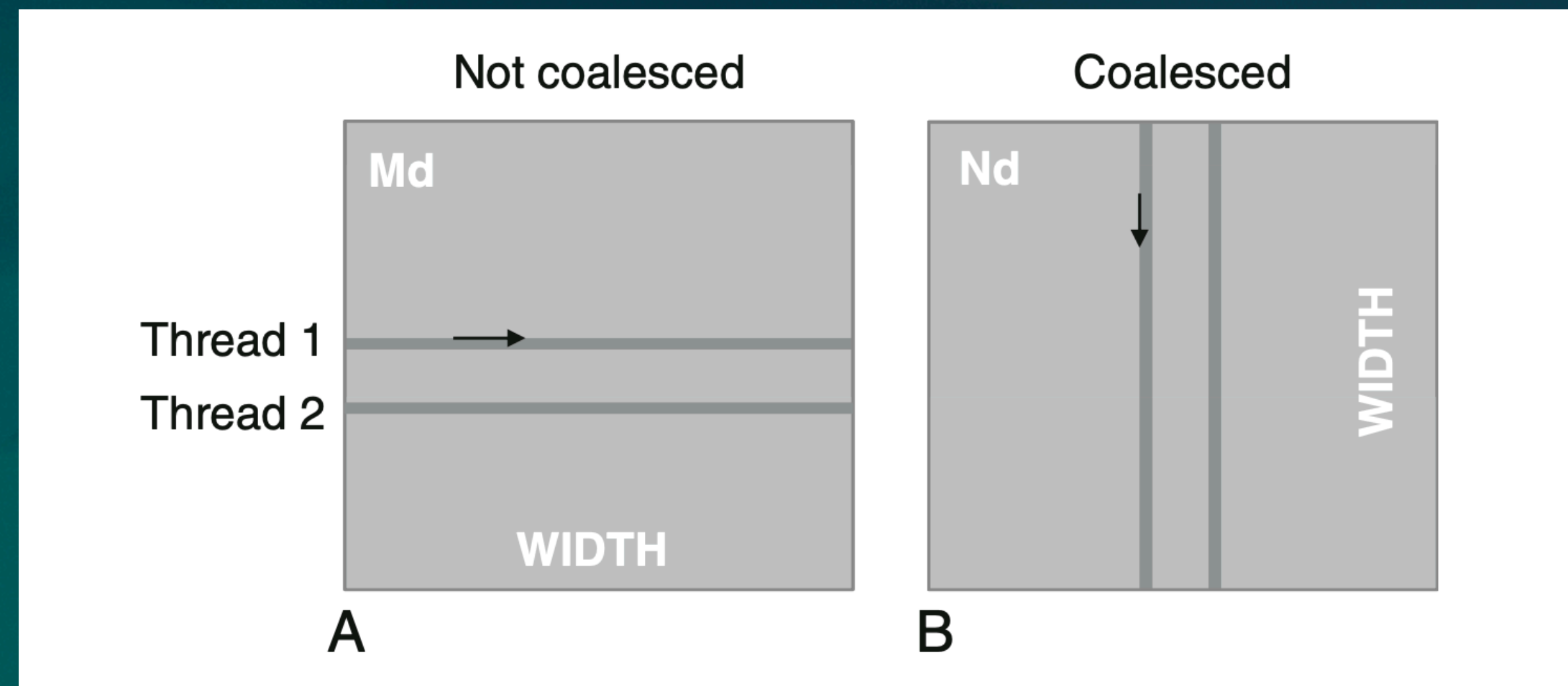
Warps

Units of Execution

- 1 warp = 32 consecutive threads (hardware)
- Single Instruction Multiple Threads (SIMT)
 - Share a program counter (rip)
- Thread divergence from if-then-else
 - Must execute both branches with masking
 - H100: New technique *warp specialization*

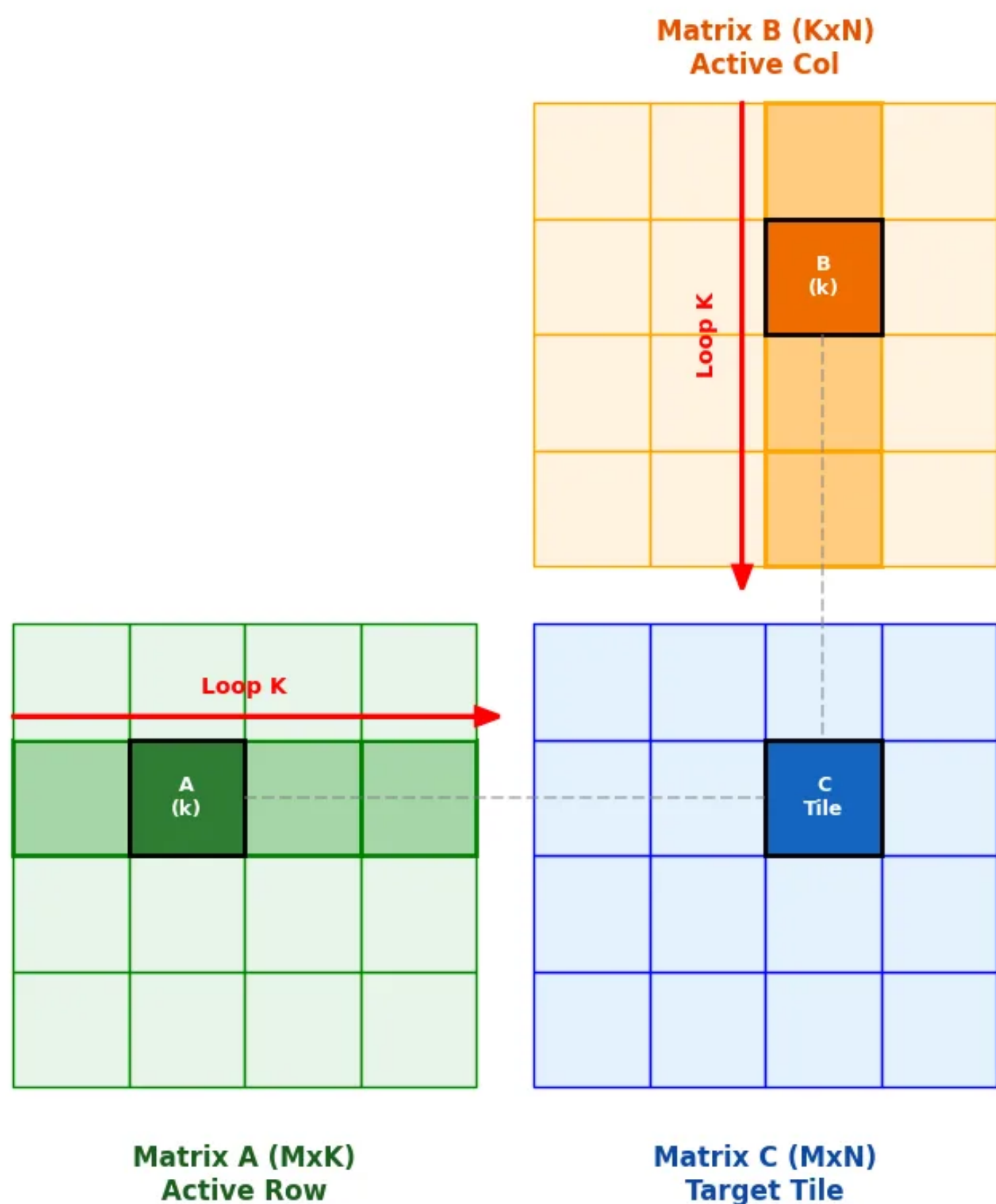
Optimization #1: Memory coalescing

- Not coalesced: row index $\leftarrow$ threadIdx.x, col index $\leftarrow$ threadIdx.y
 - **Strided** access within each warp
- Coalesced: row index $\leftarrow$ threadIdx.y, col index $\leftarrow$ threadIdx.x
 - **Sequential** access within each warp



Source: Figure 6.6, Programming Massively Parallel Processors, Kirk & Hwu

Optimization #2: Shared Memory Tiling



- For a single output element `output[row][col]`
 - Load `A[row][k_i], B[k_i][col]` { 2*k global }
- `__shared__ float As[TILE_SIZE][TILE_SIZE]`
- Reduce global memory accesses by `TILE_SIZE`
- For simplicity `TILE_SIZE == Thread Block Dimension`

Optimization #3: Data prefetching

- Tiling had two waves: loading (IO) & computation
 - Because of `__syncthreads()`, **no** IO-compute overlap

Start loading
the next tile

```
Loop {  
  
    Load current tile to shared  
    memory  
  
    __syncthreads()  
  
    Compute current tile  
  
    __syncthreads()  
}
```

A Without prefetching

Source: Figure 6.13, Programming Massively Parallel Processors, Kirk & Hwu

Optimization #3: Data prefetching

- Tiling had two waves: loading (IO) & computation
 - Because of `__syncthreads()`, **no** IO-compute overlap within threadblock

Start loading
the next tile

```
Loop {  
  
  Load current tile to shared  
  memory  
  
  __syncthreads()  
  
  Compute current tile  
  
  __syncthreads()  
}
```

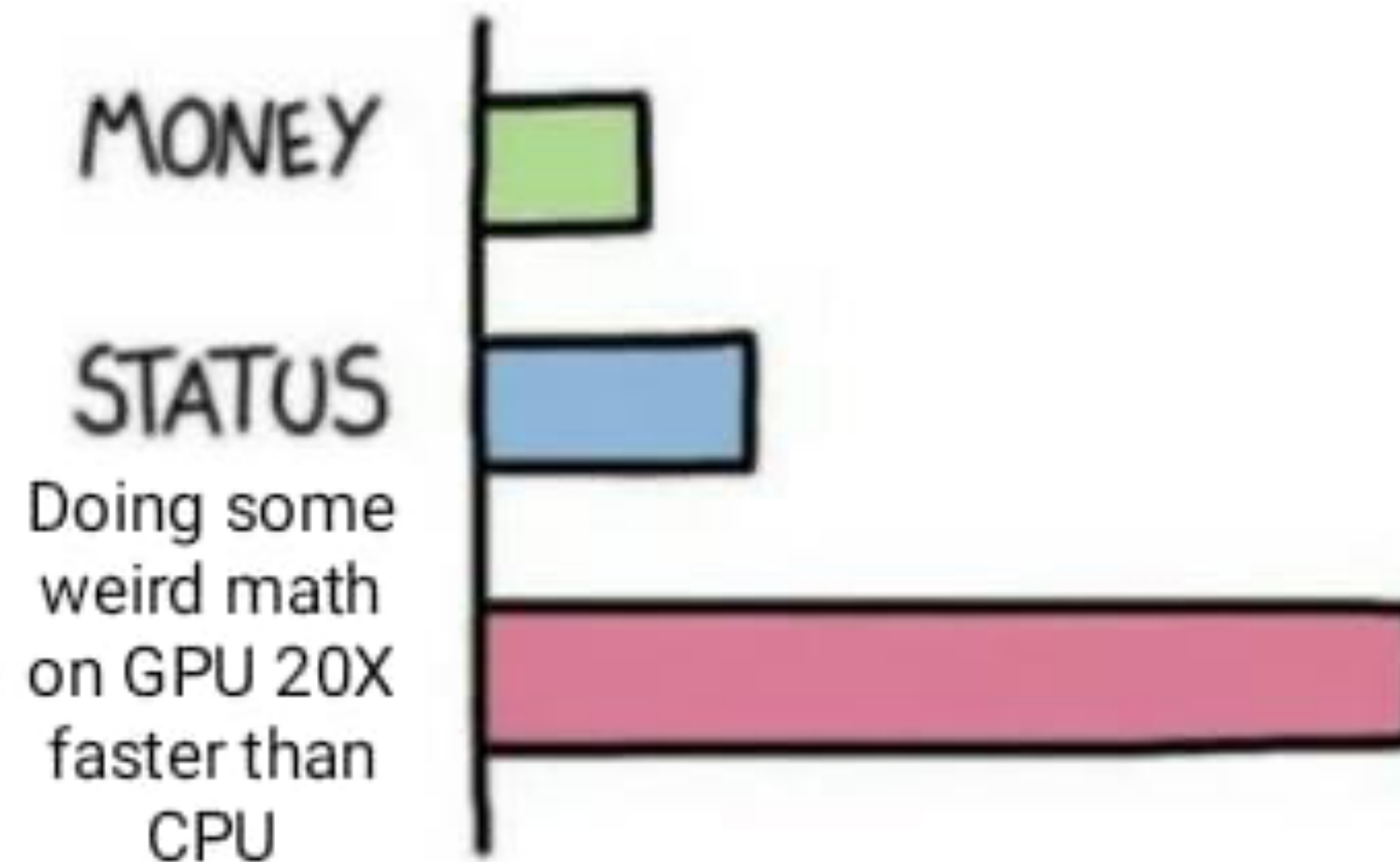
A Without prefetching

Source: Figure 6.13, Programming Massively Parallel Processors, Kirk & Hwu

```
Load first tile from global memory into  
registers  
  
Loop {  
  Deposit tile from registers to shared  
  memory  
  __syncthreads()  
  
  Load next tile from global memory into  
  registers  
  
  Compute current tile  
  
  __syncthreads()  
}
```

B With prefetching

WHAT GIVES PEOPLE FEELINGS OF POWER



Problem #3: Cumulative Sum



<https://tensara.org/problems/cumsum>

Inclusive scan Blelloch-style

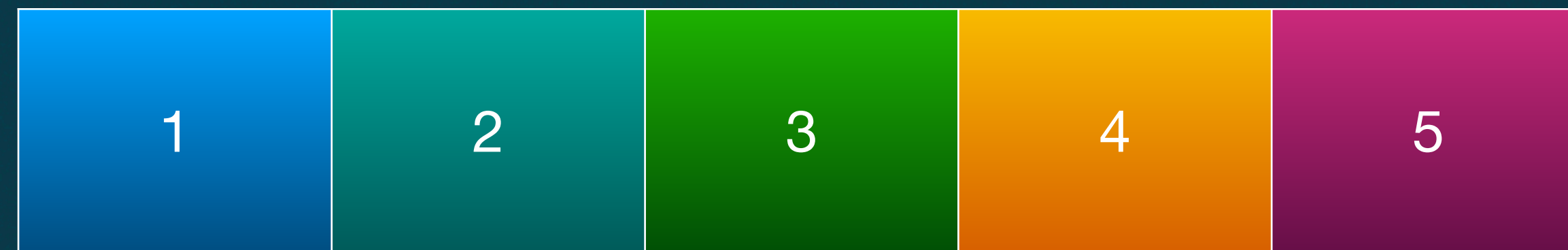
For stride = 1, 2, 4, 8, ..., blockDim:

```
local = array[threadIdx - stride]
```

```
__syncthreads()
```

```
array[threadIdx] += local
```

```
__syncthreads()
```



Problem #3: Cumulative Sum



<https://tensara.org/problems/cumsum>

Inclusive scan Blelloch-style

```
__shared__ array[blockDim]
```

```
For stride = 1, 2, 4, 8, ..., blockDim:
```

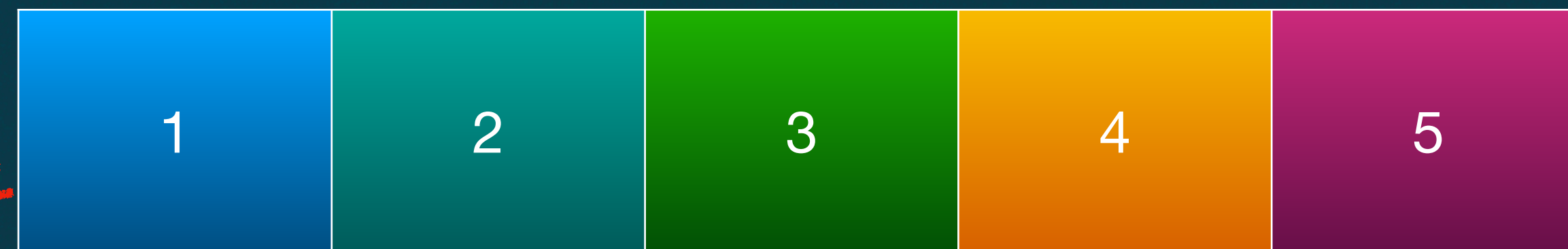
```
    local = array[threadIdx - stride]
```

```
    __syncthreads()
```

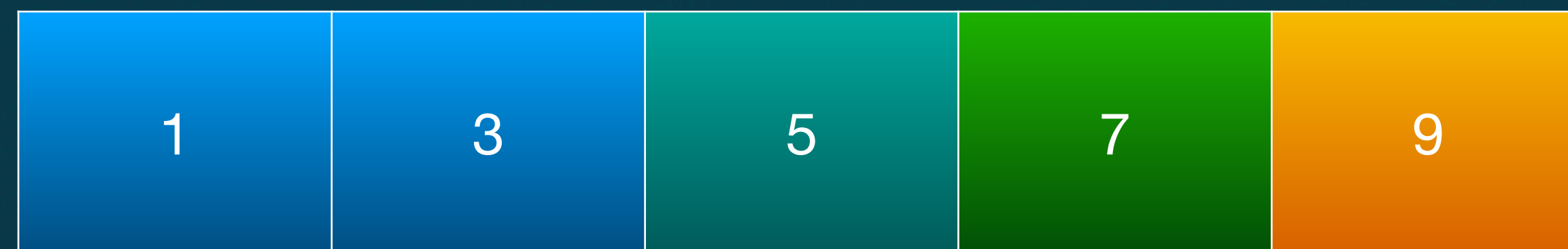
```
    array[threadIdx] += local
```

```
    __syncthreads()
```

stride



1



2



4



Bank Conflicts

- 32 banks, each 4 bytes wide (32-bit floats) *Parallelize memory accesses*
- Addresses differing by 128 bytes (0x80) map to the same bank
- Bank conflict => memory access serialized *SLOW*

Address:	0x00	0x04	0x08	0x0C	0x10	0x14	0x18	0x1C	...	0x7C
Bank:	0	1	2	3	4	5	6	7	...	31

Address:	0x80	0x84	0x88	0x8C	0x90	0x94	0x98	0x9C	...	0xFC
Bank:	0	1	2	3	4	5	6	7	...	31

Source: <https://modal.com/gpu-glossary/perf/bank-conflict>

Optimization #4 : Skew addressing

2 Banks

Mitigating bank conflicts in 1D

stride

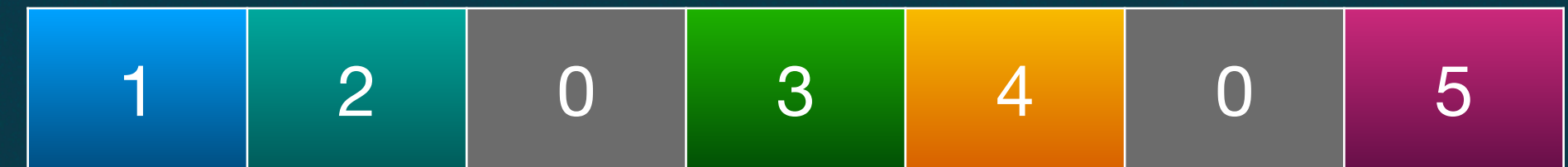
- Add padding to shared array to allow for skewing

```
shared array[blockDim +  
blockDim / NUM_BANKS]
```

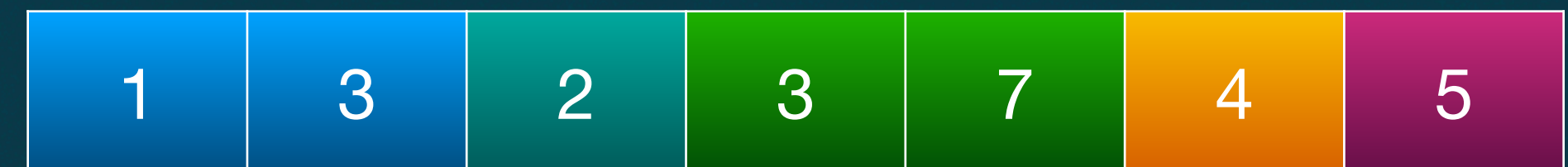
- Skew memory accesses
 - Logical -> physical mapping

```
idx |-> idx + floor(idx/NUM_BANKS)
```

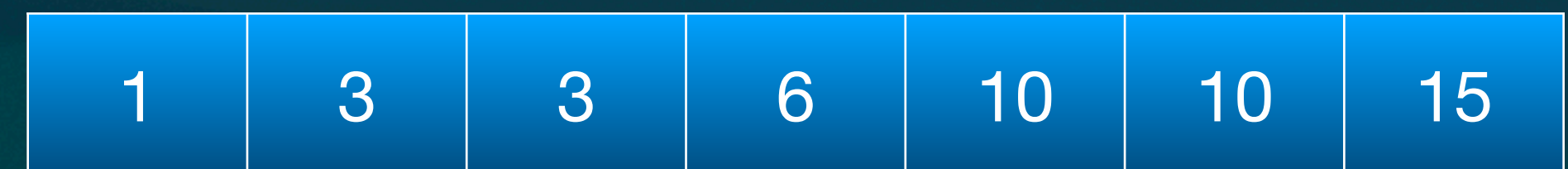
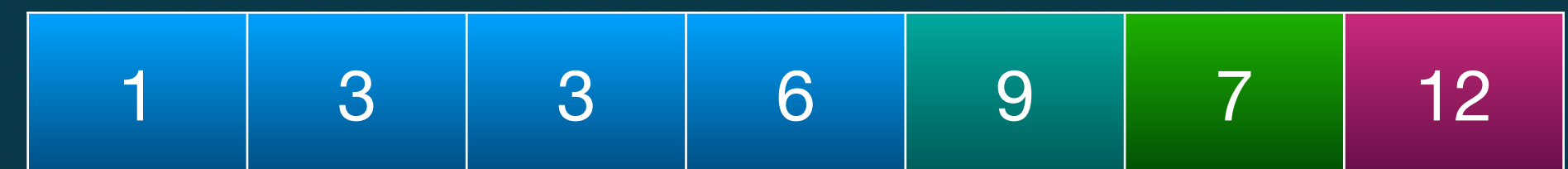
1



2



4



0

1

2

3

4

Problem #4: Tensor MatMul



<https://tensara.org/problems/matmul-3d>

Some ideas:

- Tile like GEMM, but cache K-dimension aggressively
- Pack shared memory with N tiles
- Consider batch(N)-major or M-major layout

References

- <https://modal.com/gpu-glossary>
- Programming Massively Parallel Processors, Kirk & Hwu
- <https://developer.nvidia.com/gpugems/gpugems3/part-vi-gpu-computing/chapter-39-parallel-prefix-sum-scan-cuda>